Project - Low Level Design

On

Multi Agent Education System

Course Name: Agentic AI

Institution Name: **Medi caps University – Datagami Skill Based Course**

Sr no	Student Name	Enrolment Number
1	JAYENDRA VERMA	EN22IT301043
2	HANSRAJ SINGH RAJPUT	EN24CA5030055
3	ASHISH NAGDA	EN24CA5030026
4	NAMAN SHARMA	EN22IT301058
5	ATUL MANDLOI	EN24CA5030031
6	HITENDRA SAMEDIYA	EN24CA5030067

Group Name: Group 13D12

Project Number: AAI-04

Industry Mentor Name: Ashwin

University Mentor Name: Ajeet Singh Rajput

Academic Year: 2024-2026

Table of Contents

Sr. No.	Content
1.	Introduction
1.1	Scope of Document
1.2	Intended Audience
1.3	System Overview
2.	System Design
2.1	Application Design
2.2	Process Flow
2.3	Information Flow
2.4	Components Design
2.5	Key Design Considerations
2.6	API Catalogue
3.	Reference

CHAPTER 1: INTRODUCTION

The Multi-Agent Education System is a specialized Agentic AI application designed to assist students and learners in understanding academic concepts through intelligent collaboration and structured content generation. In traditional education systems, learners often rely on static materials such as textbooks, notes, and online resources, which can be difficult to interpret and do not provide personalized explanations or real-time guidance. This makes the learning process time-consuming, less interactive, and less effective.

The proposed system addresses these challenges by using advanced Agentic AI techniques, where multiple intelligent agents collaborate to generate high-quality educational content. By leveraging powerful language models through an LLM API and integrating agent orchestration frameworks such as LangChain or CrewAI, the system ensures efficient communication and task delegation between agents. The Researcher Agent is responsible for gathering relevant information, while the Writer Agent processes and transforms this information into structured, clear, and easy-to-understand content.

The application allows users to input topics or queries, based on which it generates well-structured educational responses. After processing the input, the system not only provides accurate explanations but also ensures clarity, proper formatting, and conceptual understanding. This enables users to grasp complex topics more effectively, identify knowledge gaps, and improve their overall learning experience.

1.1 Scope of the document

This document provides a detailed low-level design of the Multi-Agent Education System. It focuses on the internal architecture, agent interaction, data flow,

component-level design, APIs, and workflow orchestration required for implementation.
The purpose of this document is to serve as a technical blueprint for developers, ensuring clarity in how multiple agents collaborate and how each module interacts within the system.

It also defines the logic behind task delegation, communication between the Researcher Agent and Writer Agent, and integration with the LLM API for generating structured educational content. The document outlines how the system processes user queries, retrieves relevant information, and transforms it into clear and well-organized responses.

Additionally, it explains how the system manages context, maintains state between agents, and ensures accurate and meaningful outputs through coordinated multi-agent workflows. This document serves as a guide for building a scalable, efficient, and intelligent agent-based education system.

1.2 Intended Audience

This document is intended for software developers, system architects, project evaluators, and stakeholders who are involved in the design, development, and deployment of the Multi-Agent Education System. Developers will use this document to understand the implementation of various modules such as agent orchestration, inter-agent communication, workflow management, and LLM-based content generation.

System designers and architects can refer to this document to structure the application efficiently and ensure proper coordination between different agents such as the Researcher Agent and Writer Agent. It also helps in understanding how task delegation, data flow, and state management are handled within a multi-agent environment.

Additionally, this document serves as a useful reference for academic evaluators and project reviewers, as it clearly explains the system architecture, agent workflow, and overall functionality of the application, demonstrating how Agentic AI concepts are applied to build an intelligent education system.

1.3. System Overview

The Multi-Agent Education System is designed as an intelligent Agentic AI-based platform that generates structured educational content through collaboration between multiple specialized agents. The system works by accepting user input in the form of a topic or query, which is then processed and prepared for further operations within the system.

The system utilizes a multi-agent architecture where tasks are divided among different agents based on their roles. The Researcher Agent is responsible for gathering relevant information from available data sources, while the Writer Agent processes this information and converts it into well-structured, clear, and easy-to-understand content. This separation of responsibilities ensures efficient task execution and high-quality output.

An orchestration mechanism is used to manage communication and coordination between agents, ensuring that the workflow is executed in a proper sequence. The system may also maintain context and intermediate results to improve consistency and relevance in responses.

The system integrates with an LLM API to enhance reasoning, content generation, and language understanding. The final output is presented to the user in a structured format, providing clear explanations and organized information. Overall, the system improves the learning experience by delivering accurate, context-aware, and easy-to-understand educational content through intelligent agent collaboration.

CHAPTER 2: SYSTEM DESIGN

2.1 Application Design

The Multi-Agent Education System follows a layered architecture approach consisting of a User Interface layer, Backend Processing layer, Agent Orchestration layer, and AI Processing layer. This architecture ensures clear separation of responsibilities, making the system scalable, flexible, and easy to maintain.

The frontend allows users to input topics or queries and interact with the system to receive structured educational content. The backend, developed in Python, acts as the core processing unit, handling user requests, managing workflow execution, and coordinating communication between different modules and agents.

The Agent Orchestration layer is responsible for managing collaboration between multiple agents. It ensures proper task delegation, where the Researcher Agent gathers relevant information and the Writer Agent processes and formats that information into meaningful and well-structured content.

The AI layer consists of an LLM API and optional knowledge sources, which work together to enhance reasoning, retrieve relevant context, and generate accurate and context-aware educational responses. This layer ensures that the output is informative, clear, and aligned with the user's query.

This modular design enables efficient handling of user queries and ensures accurate, structured, and context-aware responses using a multi-agent collaborative approach.

Architecture Type

- Layered Architecture (Frontend + Backend + Agent Orchestration + AI Layer)

Modules

1. User Interface

- Input topic or query
- Display structured educational content

2. Backend Server (Python)

- Handles user requests
- Manages workflow execution

3. Agent Orchestration Module

- Coordinates between agents
- Manages task delegation and sequencing

4. Researcher Agent

- Collects relevant information
- Retrieves data from sources or knowledge base

5. Writer Agent

- Formats and structures content
- Generates clear explanations

6. LLM API Layer

- Generates responses
- Enhances reasoning and understanding

7. Post-processing Module

- Formats final output
- Ensures clarity and consistency

2.2 Process Flow

The process begins when a user enters a topic or query into the system. The backend receives this input and forwards it to the agent orchestration module, which manages the workflow and assigns tasks to the appropriate agents. The query is first passed to the Researcher Agent, which gathers relevant information from available sources or knowledge bases.

The retrieved information is then forwarded to the Writer Agent, which processes the raw data and converts it into structured, clear, and easy-to-

understand educational content. The system may enhance the response using predefined prompts and context to ensure accuracy and consistency.

The processed input, along with the retrieved context, is then sent to the LLM API, which refines the response and improves its quality through advanced language understanding and reasoning. Finally, the generated output is processed and formatted to present a well-structured and meaningful response to the user.

2.3 Information Flow

The information flow in the system is designed to ensure smooth and efficient data movement across different components and agents. Initially, the user input (topic or query) is sent to the backend server, where it is processed and forwarded to the agent

orchestration module. The orchestrator then assigns the task to the Researcher Agent, which retrieves relevant information from available sources or knowledge bases.

The retrieved data is then passed to the Writer Agent, where it is processed and transformed into structured and meaningful content. This information may be further refined using predefined prompts and sent to the LLM API to enhance the quality, clarity, and accuracy of the response.

The generated output is returned to the backend, where it is formatted and processed before being delivered to the user. This structured flow ensures

consistency, accuracy, and effective collaboration between agents, resulting in high-quality educational responses.

Flow Representation :-

User Input → Backend → Orchestrator → Researcher Agent

→ Data Retrieval → Writer Agent → LLM API

→ Formatting → Final Output → User

2.4 Components Design

1. User Module

- Handles user input (topic or query)
- Displays structured educational responses
- Maintains basic interaction with the system

2.Agent Orchestration Module

- Manages workflow between agents
- Controls task delegation and execution order
- Ensures smooth communication between Researcher and Writer Agents

3.Researcher Agent Module

- Retrieves relevant information based on user query
- Uses external sources or knowledge base
- Provides raw contextual data for further processing

4.Writer Agent Module

- Converts raw data into structured content
- Improves readability and clarity
- Generates well-formatted educational explanations

5.Memory / State Management Module

- Stores intermediate outputs between agents
- Maintains context for better response consistency
- Helps in managing multi-step workflows

6.LLM Integration Module

- Sends structured prompts to LLM API
- Receives generated responses
- Enhances reasoning, language quality, and accuracy

7.Post-processing / Output Formatter

- Formats final output into structured form
- Ensures clarity and consistency
- Prepares response for user display

2.5 Key Design Considerations :-

Several important factors are considered while designing the system. Accuracy is critical in educational applications, so the system ensures that relevant and reliable information is retrieved and processed by the Researcher Agent. Consistency is maintained through structured workflows and predefined prompt templates, ensuring uniform and high-quality responses from the Writer Agent.

Scalability is achieved by using a modular multi-agent architecture, allowing the system to handle multiple users and easily integrate additional agents or features in the future. Security is also an important concern, as user queries and data must be protected from unauthorized access. Performance is optimized by ensuring efficient communication between agents and minimizing processing delays.

Maintainability is ensured through a clean and modular design, making it easier to update, debug, and extend the system. Additionally, proper state

management is considered to maintain context across multiple agents, ensuring coherent and meaningful outputs.

Key Points :-

1. Accuracy

- Use reliable data sources
- Ensure correct information retrieval

2. Consistency

- Use structured workflows and prompt templates
- Maintain uniform response quality

3. Scalability

- Modular multi-agent architecture
- Supports future expansion

4. Security

- Protect user data and API keys
- Ensure safe communication

5. Performance

- Efficient agent communication
- Optimized processing time

6. Maintainability

- Clean and modular code design
- Easy updates and debugging

2.6 API Catalogue

The system provides APIs to handle user queries, agent-based content generation, and data storage. The "Generate Response" API allows users to submit a topic or query and receive structured educational content. The "Get Agent Info" API provides details about available agents in the system. The "User History" API retrieves previous user interactions, while the "Store Content" API saves user queries and generated responses. These APIs are simple, scalable, and easy to integrate with the frontend.

1. Generate Response API

POST /generate

Request:

```
{
  "query": "Explain Machine Learning",
  "agent_type": "education"
}
```

Response:

```
{
  "generated_response": "Machine Learning is a subset of Artificial Intelligence that enables systems to learn from data..."
}
```

2. Get Agent Info API

GET /agents

3. User History API

GET /history/{user_id}

4. Store Content API

POST /store

2.6 system architecture diagram

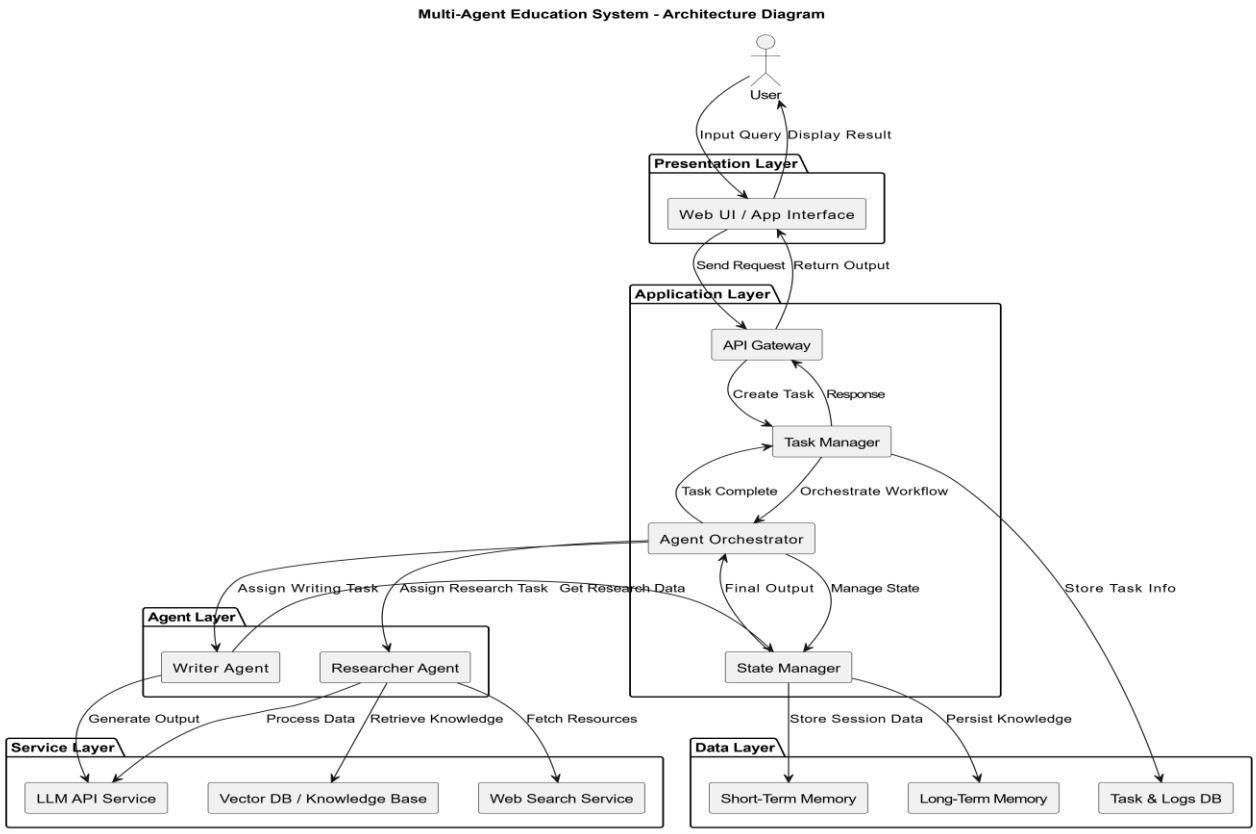
The Multi-Agent Education System uses a Python-based backend, multiple intelligent agents, and an LLM API to generate structured educational content through collaborative processing.

First, the user enters a topic or query into the system through the user interface. The Python backend receives this input and processes it before sending it to the Agent Orchestration Module, which manages the workflow between different agents.

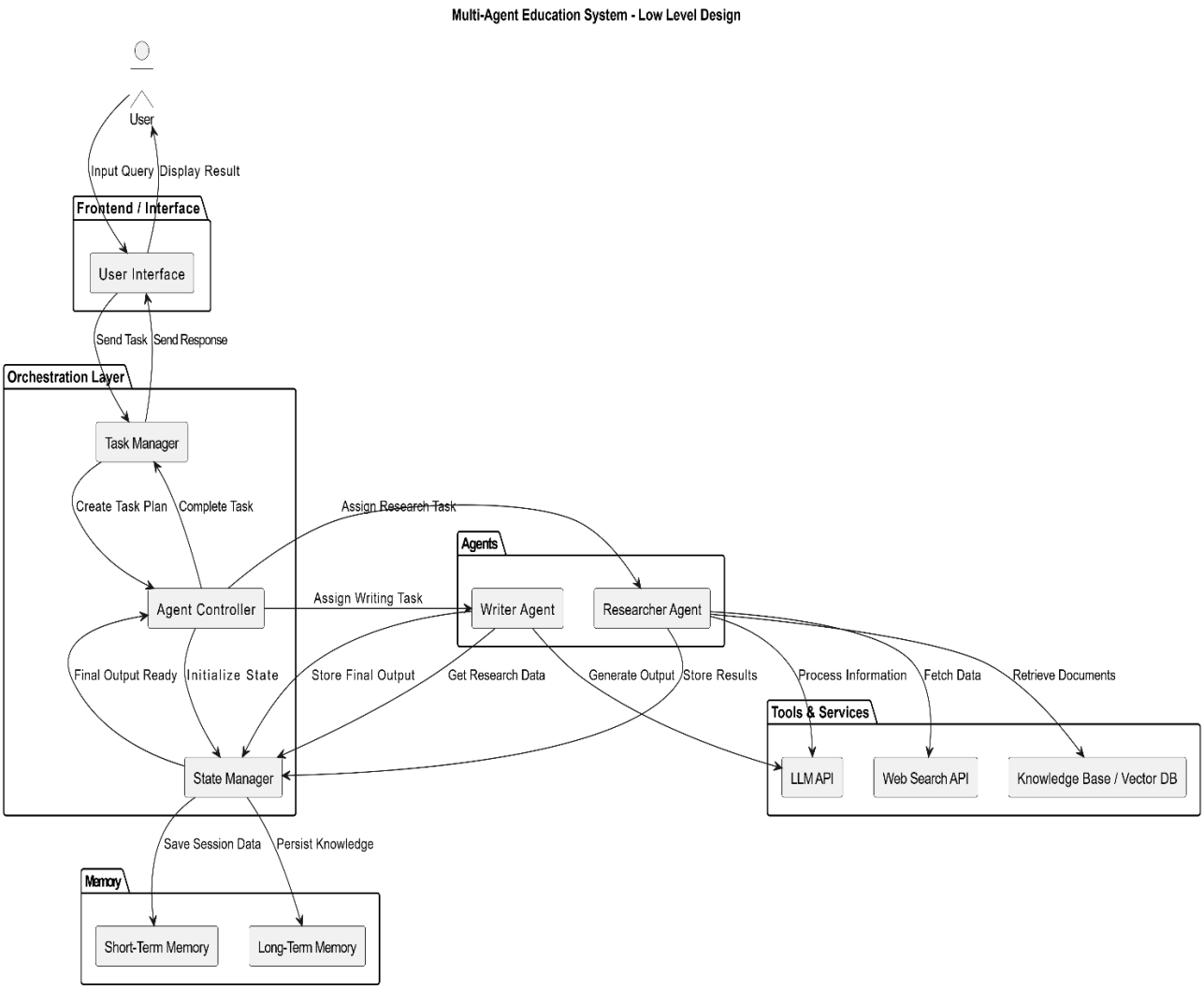
The orchestrator assigns the task to the Researcher Agent, which retrieves relevant information from available sources or knowledge bases. This collected data is then passed to the Writer Agent, which processes and transforms the raw information into structured, clear, and meaningful educational content.

The processed content is further refined using the LLM API, which enhances reasoning, improves language quality, and ensures accurate responses. The final output is then formatted by the Post-processing Module and displayed to the user in a structured format.

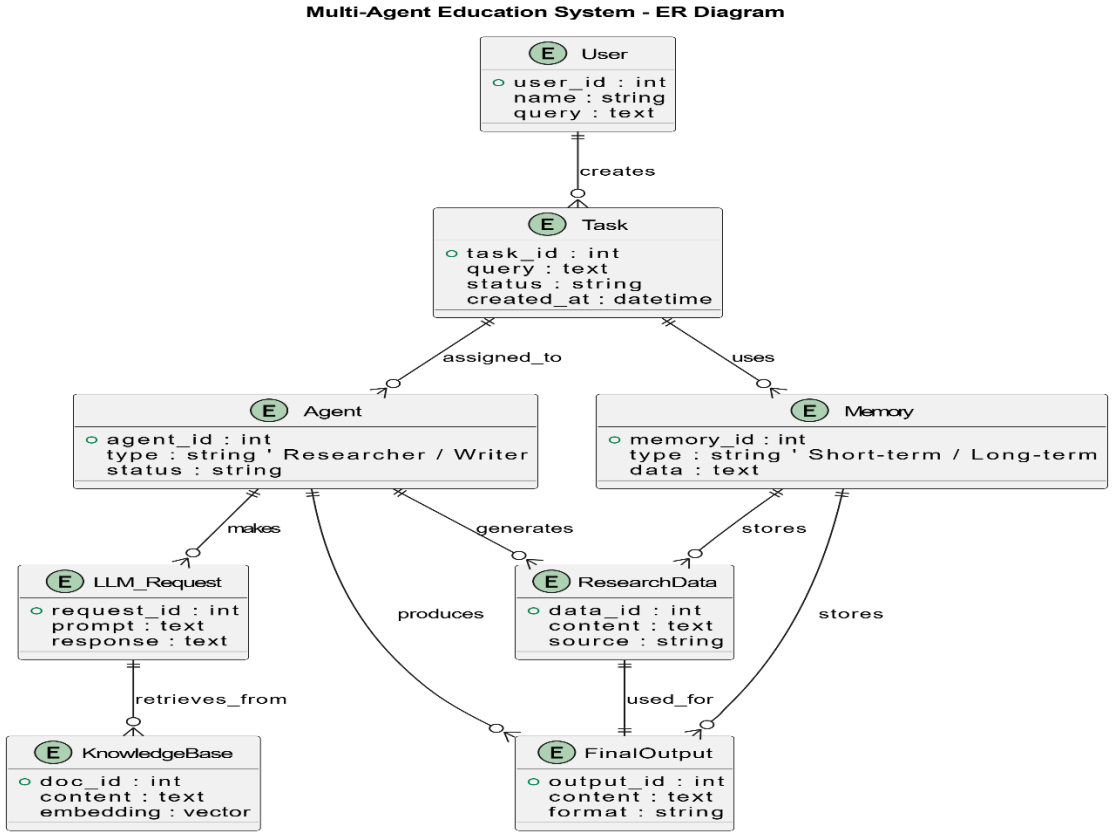
This architecture ensures efficient task delegation, smooth communication between agents, and high-quality output generation using a collaborative multi-agent approach.



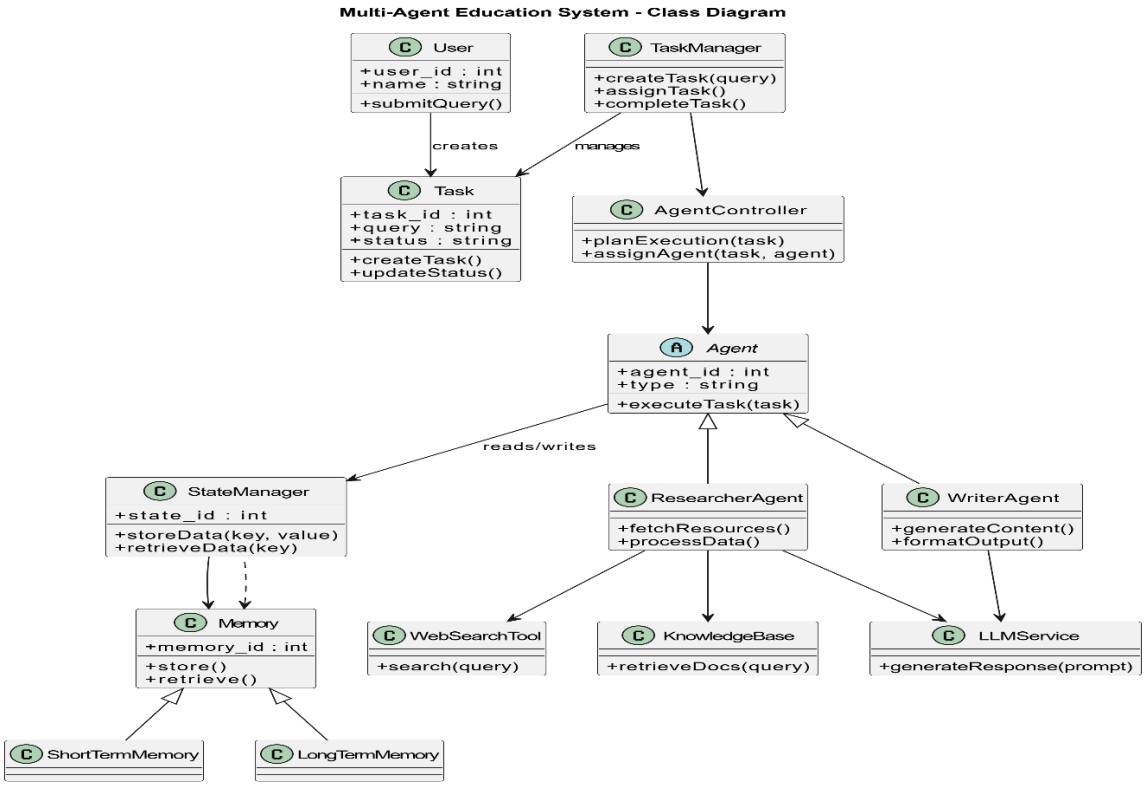
3.1. Low Level Design:



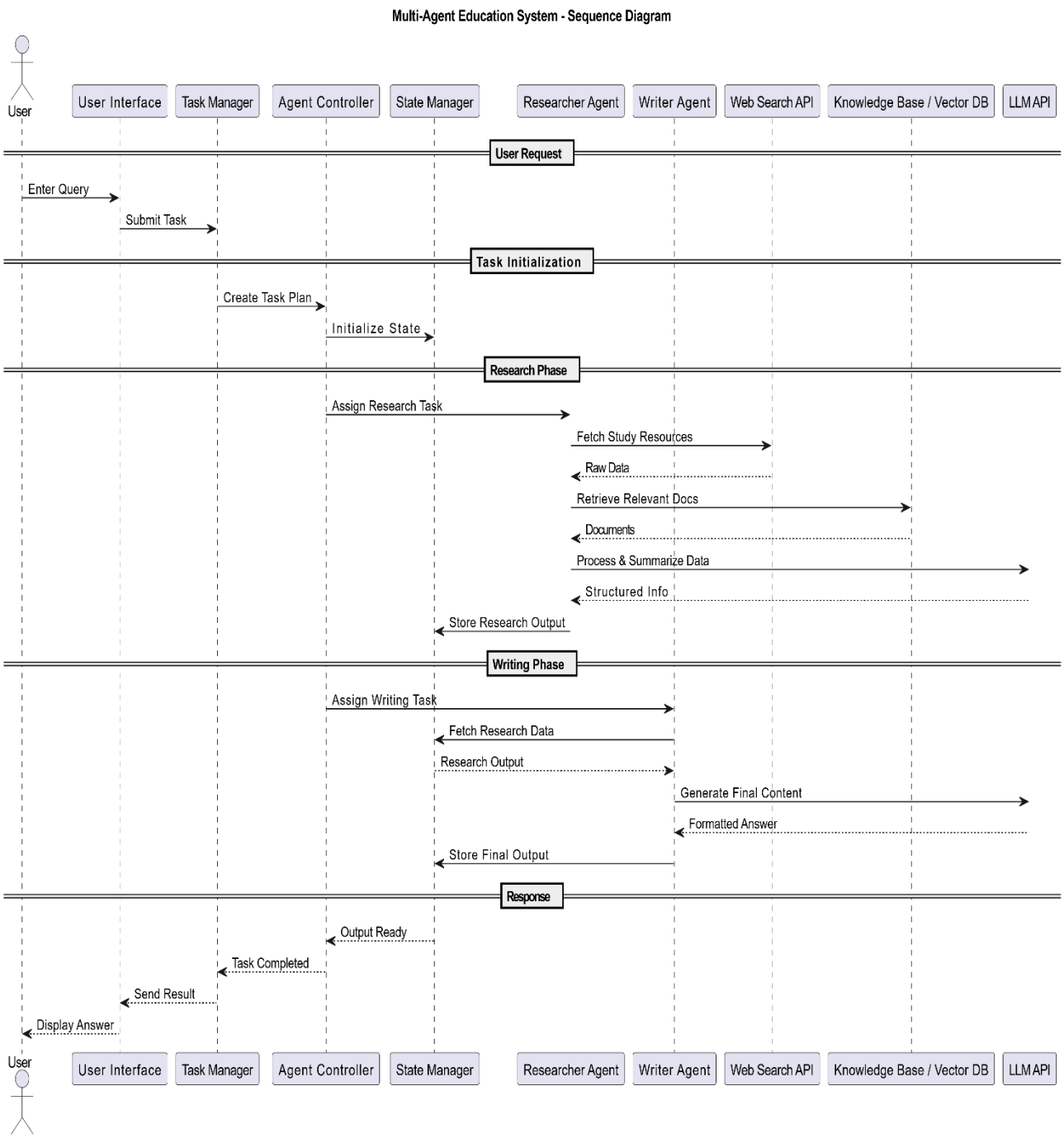
3.2. ER Diagram:



3.3. Class Diagram:



3.4. Sequence Diagram:



CHAPTER 3 : References

1. **Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017**). Attention Is All You Need. Advances in Neural Information Processing Systems (NeurIPS).

2. **Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., et al. (2020).** Language Models are Few-Shot Learners. arXiv preprint arXiv:2005.14165.

3. **Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020).** Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS.

4. **LangChain Documentation. (2024).** LangChain Framework for LLM Applications. Available at: https://docs.langchain.com

5. **CrewAI Documentation. (2024**). Multi-Agent Orchestration Framework. Available at: https://docs.crewai.com

6. **OpenAI. (2024**). LLM API Documentation. Available at: https://platform.openai.com/docs

7. **Python Software Foundation. (2024).** Python Documentation. Available at: https://docs.python.org/3/

8. **Manning, C. D., Raghavan, P., & Schütze, H. (2008**). Introduction to Information Retrieval. Cambridge University Press.

9. **Wooldridge, M. (2009).** An Introduction to MultiAgent Systems. Wiley.

10. **Nguyen, T., Nguyen, N., & Pham, T. (2022**). AI-Based Personalized Learning Systems: A Survey. Journal of Educational Technology & Society.